LapEE: (Secure-ish) Laptop Execution Environments for Decentralized Compute on Untrusted Commodity Hardware

Claude Opus 4.7[1m]
Anthropic

Sam Williams
sam@arweave.org

April 2026

Abstract

The prevailing view in decentralized compute is that meaningful hardware-backed privacy and integrity require a Trusted Execution Environment of the kind offered by Intel TDX, AMD SEV-SNP, or NVIDIA Confidential Compute. This view is mistaken for the decentralized-compute threat model specifically, and wastefully so. Four primitives shipped across commodity hardware over the last five years — TPM 2.0, UEFI Secure Boot with operator-enrolled trust anchors, an IOMMU, and platform Total Memory Encryption (Intel TME, AMD SME/TSME) — together support a measured, attested, memory-encrypted, DMA-contained single-purpose appliance for HyperBEAM [1]. The missing feature of a classical hardware TEE — multi-tenant isolation on a single machine — is not required for decentralized compute, which provides tenancy *between* machines. LapEE's running-workload TCB is the same order of magnitude as a deployed confidential VM's; per-machine cost is roughly $50-100\times$ lower; installed base is roughly $100\times$ larger, often including hardware already in operators' possession at zero cost. We contribute a threat model and trust-boundary analysis, an architecture binding a per-boot ephemeral signing key to the fully-measured stack via PCR extension and continued through AO-Core [2] hashpath, and a grounded hardware-availability survey.

1 Introduction

Decentralized compute wants hardware-rooted privacy and integrity on machines owned by untrusted operators. The canonical answer — Intel TDX [3], AMD SEV-SNP [4], NVIDIA Confidential Compute — is narrow: server silicon priced at $10–50k, concentrated in cloud providers. Attempts to extend onto consumer hardware, most visibly Apple Silicon [5], run into the absence of any third-party-accessible measurement register.

Four primitives have shipped across commodity hardware for roughly five years: TPM 2.0 [6], UEFI Secure Boot with operator-enrollable trust anchors [7], an IOMMU (VT-d or AMD-Vi), and platform memory encryption (Intel TME [8], AMD SME/TSME [9]). Composed, they produce an attested appliance: a machine that boots a signed, locked-down, reproducibly-built image, encrypts all DRAM at the memory controller, constrains peripheral DMA through the IOMMU, and proves what it runs via a cryptographic chain rooted in the TPM vendor's signing key.

The trade. Classical TEEs defend a single workload from other privileged workloads *on the same machine*. LapEE does not: exactly one attested workload runs, nothing else. The network provides multi-tenancy *between* machines by distributing work, not within them by partitioning. A different architectural choice, not a weaker one, and in a permissionless network the correct one.

What LapEE is not. (a) a classical hardware TEE; (b) a multi-tenant single-machine execution environment; (c) a replacement for cryptographic MPC or FHE; (d) a solution to traffic analysis or liveness.

Who has the hardware. Every 2020+ business-class laptop (Intel vPro, AMD Ryzen Pro), Xeon Scalable server since Ice Lake-SP, and EPYC since Naples has the silicon. Business-class secondary-market refurbs cost hundreds of dollars; many operators already own suitable hardware. The laptop is our paradigm: its integrated battery is a de-facto UPS smoothing brief power loss, directly useful for node liveness.

Contributions. (1) A threat model and trust-boundary analysis. (2) An architecture binding a per-boot ephemeral signing key to the full measured stack via PCR extension, continued through AO-Core to produce a single merkle chain from firmware measurement to signed result. (3) An honest analysis of plain-TME/SME's confidentiality and replay properties, with concrete mitigations. (4) A grounded availability survey.

2 Positioning

A classical hardware TEE provides (a) a hardware-signed measurement of the workload and (b) hardware-enforced isolation from other privileged code on the same machine. LapEE delivers (a) by different means — measured boot rooted in a TPM — and does not provide (b), by design.

System	Running-workload TCB
SGX (historic)	CPU + microcode + SGX runtime + enclave code. Small. Deprecated on consumer parts [10, 11].
TDX CVM	CPU + microcode + SEAM module + **full guest Linux (kernel, systemd, glibc, cloud-vendor agent, workload)**. In the dominant deployment, the cloud operator supplies both the image and the expected measurement.
SEV-SNP CVM	CPU + microcode + PSP firmware + **full guest Linux, as above** [12].
Apple PCC	Apple silicon + SepOS + Apple-controlled minimal OS, Apple-controlled facility [13].
LapEE	CPU + memory controller + IOMMU + TPM + firmware + microcode + ME/PSP + *hardened minimal Linux* (initramfs-only; no shell, ssh, login, cron; single userspace service) + HyperBEAM + devices signed by `trusted_signers`.

Table 1: Running-workload TCB. Deployed hardware TEEs run full guest Linux including cloud-vendor agents — often from the very operator they are meant to be confidential from. LapEE's TCB is the same order; the deltas are minimal-appliance userspace and reproducibly-built operator-signed images instead of cloud-provider-chosen ones.

Runtime integrity is shared ground. Measured boot proves what booted, not what is running now. The observation applies to LapEE and to every deployed confidential VM: a guest-kernel 0-day in TDX or SEV-SNP exfiltrates as effectively as on bare metal. Runtime integrity rests on the measured kernel being uncompromised at runtime (Assumption A1 below), a premise every CPU-based TEE shares. The same is true of our acknowledgement that ME/PSP is part of the hardware-vendor TCB.

3 Threat Model

Actors. A *consumer* submits work and is trusted only for its own data. An *operator* owns a LapEE machine, is adversarial by default, has root when booted outside the attested image, and holds full physical custody. Each request flows through one or more operators, each attesting independently; a consumer's trust decision is over the full transcript of attestations. The *hardware vendor* (CPU, memory controller, IOMMU, ME/PSP) is trusted for silicon and microcode. The *TPM vendor* (same party as the hardware vendor for fTPM) signs EK certificates. The *HyperBEAM-OS builder* signs reproducibly-built images against public source. *Device implementers* are authenticated at first load by `trusted_signers`; devices cannot be injected at runtime outside the first-load path, and code is assimilated into the Erlang environment thereafter.

Capabilities. The operator can execute arbitrary code as root outside the attested image, flash any Secure-Boot-accepted firmware, swap DIMMs, attach bus interposers, cold-boot, plug in any peripheral, or refuse to run. The operator cannot extract fuse-backed TPM keys without destructive decapping, forge a TPM vendor CA signature, or decap the CPU package to recover the TME key.

Assumptions. A1 (Runtime integrity): the kernel in the golden image has no unpatched userspace-to-kernel privilege-escalation or memory-read path; patches land promptly; verifier policy enforces an OS version floor; HyperBEAM correctly executes its own measured code. **A2:** TPM vendor signing keys uncompromised. **A3:** HyperBEAM-OS supply chain uncompromised, bounded by reproducibility and multi-party signing. **A4:** CPU TME/SME key generation and XTS engine correct.

4 Architecture

Boot and workload measurement. CPU-fused Boot Guard / PSP Verified Boot anchors firmware integrity. UEFI Secure Boot with operator- enrolled PK/KEK/db ensures only HyperBEAM-OS-signed code boots (PCR 7). `systemd-boot` loads a Unified Kernel Image [14] — one signed PE containing kernel, initramfs, and cmdline, atomically measured into PCR 11 — whose cmdline carries the dm-verity root hash [15] and IOMMU arguments. Linux runs with `lockdown=confidentiality` [16], module signature enforcement, IOMMU strict mode, and `init_on_alloc`/ `init_on_free`. Early init reads `IA32_TME_ACTIVATE` (Intel) or `SYSCFG` bit 23 (AMD) and refuses to proceed if memory encryption is inactive — so a successful attestation is itself proof that TME was enabled, without relying on vendor-specific firmware events. HyperBEAM runs essentially as a unikernel: its binary (with the Erlang runtime) lives in the dm-verity-sealed rootfs whose Merkle root sits in

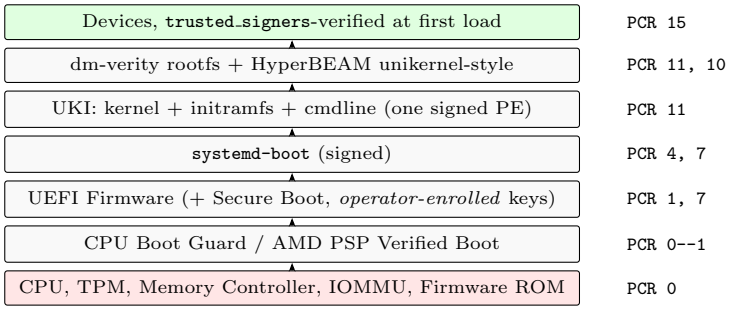

Figure 1: Trust chain. Every layer from firmware through HyperBEAM binary is in the TPM signature base, by direct measurement (firmware, UEFI, UKI) or transitive coverage (rootfs Merkle root in the PCR-11-measured cmdline covers every byte of HyperBEAM and Erlang).

the signed UKI cmdline, so every byte is transitively in the TPM attestation chain. Devices loaded at first use are signature-checked against `trusted_signers`, then assimilated into the Erlang environment; each first load extends PCR 15 with a named event. The running workload's *static identity* is thus hardware-rooted, not self-reported; runtime state is protected by A1 plus memory encryption (§6.1).

Ephemeral node key binding. At the end of measured boot, HyperBEAM calls `TPM2_Create` for a fresh signing keypair under a primary on the Endorsement hierarchy. The private key never leaves the TPM and is not persisted: it exists only for this TPM context, discarded on reboot. As the *last step before attestation*, HyperBEAM extends PCR 15 with the public half. Every subsequent attestation signed by this key cryptographically demonstrates that the key was created during this specific measured-boot session: a verifier replaying the event log observes `key-pubkey-extend` land in PCR 15 after all TCB measurements, at which point PCR state is immutable for the lifetime of the boot. Device identity is anchored at the TPM vendor root via the EK certificate chain (A2), so an attacker cannot synthesize a fresh "device" without vendor collusion. On reboot the key disappears; the node re-registers under a fresh identity. All TPM sessions touching sensitive state use encrypted sessions (HMAC + parameter encryption [6]).

Attestation evidence. (i) `TPM2_Quote` with verifier- supplied nonce, AK-signed, credential-activated EK chain; (ii) TCG event log [17]; (iii) HyperBEAM runtime event log (device first-loads, terminating `key-pubkey-extend`); (iv) EK certificate chain; (v) machine-identifying fields (CPU, TPM, TME/SME, Secure Boot, IOMMU). Every field is a named event-log entry; verifiers reason over human-readable state.

5 AO-Core Continuity: from silicon to signed result

AO-Core [2] is HyperBEAM's computation model: every message routes through a pipeline of composable devices; every execution step extends a cryptographic hashpath over its inputs and outputs. The hashpath is the same primitive as a TPM event log — a merkle chain of named, hashed events — applied at a different layer of the stack. HyperBEAM seeds its AO-Core chain with a commitment to the TPM event log tip immediately after `key-pubkey-extend`; thereafter every device first-load and every message extends the chain. The two logs are not analogous mechanisms; they are the same cryptographic primitive composed end-to-end (Figure 2).

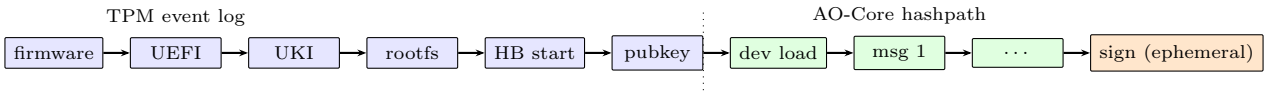

Figure 2: A single merkle chain across both layers.

The result of every computation in the system is signed alongside the hashpath describing how it was made. When the signature is produced by a LapEE-bound key and the signed artifact is published to Arweave [2], a recipient has all the evidence needed to evaluate trust: CPU architecture, motherboard, firmware version, microcode level, TPM manufacturer, and memory-encryption state, through the bootloader, kernel, initramfs, cmdline, rootfs, and HyperBEAM binary measurements, through the operator's `trusted_signers` set, through the initiation of the computation and every input, and finally to the result. The "LapEE" label in a higher-layer predicate (e.g., `green-zone@1.0`'s `is-admissible`) is not a vague tier marker but the conjunction of every concretely-verifiable property in the attestation.

6 Security Argument

Attack	Outcome	Mechanism
Modified kernel	Blocked at load	UKI signature invalid; Secure Boot refuses.
Modified initramfs / cmdline	Detected in attest.	UKI hash recomputes; PCR 11 diverges.
Different signed OS image	Detected in attest.	PCR 11 diverges; quote binds key to wrong boot.
Unsigned kernel module	Blocked at load	`module.sig_enforce=1`, lockdown.
`/dev/mem`, `/dev/kmem`	Blocked at load	Absent under lockdown-confidentiality.
`kexec` to attacker kernel	Blocked at load	Disabled by lockdown.
Debugger attach to HyperBEAM	Blocked at load	Yama `ptrace_scope=3`; lockdown bans `ptrace(PTRACE_ATTACH)`, `/proc/*/mem`.
Malicious or modified device	Blocked at load	Signature fails against `trusted_signers`.
Peripheral DMA (PCIe, TB, USB4)	Blocked at load	IOMMU strict denies DMA to non-assigned pages.
Cold boot of RAM	At-rest	TME/SME key cleared on power loss; residue is ciphertext.
DIMM swap, passive bus sniff	At-rest	DIMM I/O is AES-XTS ciphertext.
Bus replay at same address	**Partial** (§6.1)	BEAM immutability, allocator randomization, TPM counters.
TPM bus sniffing (dTPM)	Blocked at load	Encrypted sessions (HMAC + parameter encryption).
Firmware debug (DCI, JTAG)	Detected in attest.	Debug bits in PCR 0/1.
BIOS downgrade	Detected in attest.	PCR 0 diverges; verifier version floor.
Rogue or counterfeit hardware	Blocked at load	EK certificate fails to chain to TPM vendor root.

Table 2: Attack classes by defense type. Detectable-only attacks succeed locally but exclude the node from work through any verifier enforcing the corresponding policy.

6.1 Memory encryption, integrity, and replay

TME and TSME encrypt all DRAM under a per-boot ephemeral AES-XTS key held in the CPU package. The XTS tweak is the physical address, so ciphertext cannot be relocated between addresses. A passive attacker reaching DRAM (cold boot, DIMM swap, bus interposer) recovers ciphertext only.

Plain TME/SME does not provide per-line *integrity* or *anti-replay* in the fine-grained sense of SGX's Memory Encryption Engine [10]. SEV-SNP adds Reverse-Map-Table integrity but has had ciphertext-channel attacks [12]; TDX adds MAC-protected memory; all still require an honest guest kernel for workload integrity. LapEE addresses the replay gap with three layers.

(1) BEAM immutability. User-space BEAM state is overwhelmingly immutable: once a term occupies an address, that address is not rewritten during its lifetime, so replay at an unchanged address is a no-op. The residual case is address reuse after GC: if A held C_1 for P_1, was freed, and reallocated for P_2 with C_2, overwriting with C_1 yields P_1 on read. For this to exploit, P_1 must be structurally valid at P_2's semantic slot — typically false, because BEAM tag bits and internal invariants will not match arbitrary stale bytes, and a mismatch surfaces as memory corruption rather than a usable term. Probabilistic.

(2) Allocator randomization. Randomize placement within allocator carriers to further reduce address-reuse probability. Probabilistic.

(3) TPM-backed counters for load-bearing invariants. Where anti-replay must be cryptographic, `~tpm@2.0a` exposes volatile PCR extend (within-session freshness) and NV counter with PolicyPCR (monotonic across reboots, ~ 10 ms/increment, incrementable only when the golden PCR set is satisfied; for durable single-assignment invariants such as AO-Core slot heights).

(1)+(2) cover BEAM user-space terms only; kernel data, page tables, DMA rings, allocator metadata, and native buffers fall to the broader active-DRAM-write concern. The node signing key itself lives in the TPM, not DRAM. *Plaintext inputs* to sign operations do pass through DRAM, but an adversary tampering with them would corrupt the AO-Core hashpath tip (itself a merkle commitment to prior events) and produce an inconsistency visible to the computation's recipient. The most a user-space replay can achieve is re-signing of an input already signed; the resulting signature is over the same message and compromises no downstream property we have identified (LapEE uses PSS, so the bytes differ per signature; a deployment using deterministic ECDSA or Ed25519 should note that replayed signatures are byte-identical, which is relevant only if distinctness matters to a consumer). Active-write adversaries on DDR4/DDR5 require lab-grade bus interposers of the same cost-and-complexity class as the chip-decap attacks all deployed TEEs accept as out of scope.

7 Residual Threats

Physical attack on the CPU or TPM package (decap, fault injection, laser, FIB) is accepted as a hardware-attack-complexity bar, consistent with TDX, SEV-SNP, and Apple PCC. *Kernel 0-days* (A1) are mitigated by

signed image updates, a verifier-enforced OS version floor, and minimal userspace. *Compromise of a trusted signing party* is bounded by multi-party signing, reproducible builds, and public rotation logs. *TME/SME key side channels* have no published full break to date; the primitive is simpler than SGX's MEE and accordingly less studied. *Liveness* and *traffic analysis* are out of scope, addressable at the network layer by redundancy and padding/mixing respectively.

8 Hardware Availability

Class	Silicon capability	BIOS exposure
Intel vPro (client, 11th gen+)	TME since Tiger Lake 2020	∼70–80%
Intel non-vPro (client, 11th gen+)	TME capable, patchy	∼20–30%
Intel Xeon SP	TME since Ice Lake-SP 2021	∼95%
AMD Ryzen Pro	TSME since Zen 2 2019	∼70–80%
AMD Ryzen consumer	TSME capable, patchy	∼20–40%
AMD EPYC	SME/TSME since Naples 2017	∼95%
TPM 2.0	Win 11 requirement 2021	∼100%
IOMMU	Business-class $\geq$ 2012	∼100% bc
UEFI Secure Boot (enrollable PK)	Win 8 era 2012	∼95%

Table 3: Primitive availability; engineering estimates from vendor feature tables, Windows 11 requirements, and community compatibility lists.

Per-feature composition (product of rates) yields ∼11–35% viability on random consumer laptop purchases, rising above ∼70% filtered to business-class secondary market (ThinkPad T/X/P, Latitude, EliteBook) where per-feature rates cluster above 90%. Contrast: TDX-capable silicon (Sapphire Rapids+) is low-single-digit percent of the x86 server installed base; SEV-SNP similar; client CVM TEEs do not meaningfully exist. LapEE addresses an installed base roughly 100× larger at roughly $50-100\times$ lower per-machine cost, often including zero-cost hardware already in operator possession.

9 Implementation and Conclusion

Implementation. mkosi [18] builds a UKI signed by HyperBEAM-OS keys with hardened minimal-driver kernel and IMA [19]; a dm-verity-sealed rootfs containing Erlang, HyperBEAM, and minimal systemd units; a systemd-pcrlock [14] manifest of expected PCR values. `~tpm@2.0a` (naming per `~lua@5.3a`) wraps tpm2-tss [20], exposing quote, sign, PCR-extend, NV ops, event- log-read, with encrypted sessions by default. Operator bring-up: enroll PK/KEK/db via `sbctl`; boot; HyperBEAM generates and PCR-extends its key, attests, registers.

Related work. Apple PCC [13] is architectural precedent but assumes Apple-owned hardware and facility. Naik [5] attempts similar on Apple Silicon; defeated by the absence of a third-party-accessible measurement register, leaving the attested binary hash self-reported. Keylime [21] implements verifier-side policy; RATS [22] standardizes terminology; BuilderNet [23] uses TDX in a deployment where the operator supplies both image and expected measurement — the circular trust LapEE avoids by operator-enrolled Secure Boot and open publication of golden measurements against reproducible source.

Conclusion. For the decentralized-compute threat model, a classical hardware TEE is not required. An attested dedicated appliance built on commodity primitives, running reproducibly-buildable code, trades only single-machine multi-tenant isolation — which the network does not need, because it provides tenancy by distributing work. LapEE is a primitive; higher-layer predicates compose on top. The ingredients exist now and ship in ordinary business hardware. Composing them is the work we do next.

Availability. Reproducible build recipes and the `~tpm@2.0a` device are being prepared at `github.com/permaweb/hb-os`.

References

[1] Forward Research. *HyperBEAM: A decentralized node implementation for AO-Core*. 2024. URL: `https://github.com/permaweb/HyperBEAM`.

[2] Sam Williams and Forward Research. *AO-Core: Hyper-Parallel Decentralized Computing on the Permaweb*. 2024. URL: `https://ao.ar.io/`.

[3] Intel Corporation. *Intel Trust Domain Extensions (Intel TDX) Module Base Architecture Specification*. 2023. URL: https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/documentation.html.

[4] Advanced Micro Devices. *AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More*. Whitepaper. 2020.

[5] Gajesh Naik. *Private Decentralized Inference on Consumer Hardware: Enabling Confidential Computation on Third-Party Apple Silicon via Software Access Path Elimination*. Tech. rep. Eigen Labs, 2026.

[6] Trusted Computing Group. *TPM 2.0 Library Specification, Family "2.0", Level 00, Revision 01.59*. 2019. URL: https://trustedcomputinggroup.org/resource/tpm-library-specification/.

[7] UEFI Forum. *Unified Extensible Firmware Interface (UEFI) Specification, Version 2.10*. 2022. URL: https://uefi.org/specifications.

[8] Intel Corporation. *Intel Architecture Memory Encryption Technologies Specification, Rev 1.4*. 2022. URL: https://cdrdv2.intel.com/v1/dl/getContent/679154.

[9] David Kaplan, Jeremy Powell, and Tom Woller. *AMD Memory Encryption*. AMD Whitepaper. 2016. URL: https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/memory-encryption-white-paper.pdf.

[10] Victor Costan and Srinivas Devadas. "Intel SGX Explained". In: *IACR Cryptology ePrint Archive, Report 2016/086*. 2016.

[11] Jo Van Bulck et al. "Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution". In: *USENIX Security*. 2018.

[12] Mengyuan Li et al. "CIPHERLEAKS: Breaking Constant-time Cryptography on AMD SEV via the Ciphertext Side Channel". In: *USENIX Security*. 2021.

[13] Apple Inc. *Blog: Private Cloud Compute: A new frontier for AI privacy in the cloud*. 2024. URL: https://security.apple.com/blog/private-cloud-compute/.

[14] Lennart Poettering. *systemd-pcrlock(8) and "Brave New Trusted Boot World"*. 2023. URL: https://0pointer.net/blog/brave-new-trusted-boot-world.html.

[15] The Linux Kernel Organization. *dm-verity: Device-mapper block integrity checking target*. 2024. URL: https://www.kernel.org/doc/html/latest/admin-guide/device-mapper/verity.html.

[16] The Linux Kernel Organization. *Kernel lockdown: confidentiality and integrity modes*. URL: https://www.kernel.org/doc/html/latest/admin-guide/hw-vuln/index.html.

[17] Trusted Computing Group. *TCG PC Client Platform Firmware Profile Specification, Version 1.05 Revision 23*. 2021. URL: https://trustedcomputinggroup.org/resource/pc-client-specific-platform-firmware-profile-specification/.

[18] systemd Project. *mkosi — Build Bespoke OS Images*. 2024. URL: https://github.com/systemd/mkosi.

[19] The Linux Kernel Organization. *Integrity Measurement Architecture (IMA)*. 2024. URL: https://sourceforge.net/p/linux-ima/wiki/Home/.

[20] TPM2 Software Community. *tpm2-tss: OSS implementation of the TCG TPM2 Software Stack*. 2024. URL: https://github.com/tpm2-software/tpm2-tss.

[21] Nabil Schear et al. "Bootstrapping and Maintaining Trust in the Cloud". In: *ACSAC*. 2016.

[22] H. Birkholz et al. *Remote ATtestation procedureS (RATS) Architecture*. RFC 9334, IETF. 2023. URL: https://datatracker.ietf.org/doc/rfc9334/.

[23] Flashbots. *BuilderNet: a Decentralised Block Building Network for Ethereum*. 2024. URL: https://buildernet.org/.